

P0301R0: Wording for Unified Call Syntax

Split and change 5.2.2 [expr.call] paragraph 1:

... [Note: ...]

If a function or member function name is used, the name can be overloaded (Clause 13), in which case the appropriate function shall be selected according to the rules in 13.3 [over.match]. **If**

- the *postfix-expression* is an unparenthesized *unqualified-id*,
- lookup for that *unqualified-id* found no names or overload resolution found no viable functions (13.3.2 [over.match.viable]),
- there is at least one argument, and
- the first argument has class type,

then the function call expression is equivalent to a function call expression whose postfix expression is a class member access expression (5.2.5 [class.mem]) with the first argument as the object expression and the *unqualified-id* (see above) as the *id-expression*, and whose arguments are the remaining arguments (if any) of the original call. [Example:

```
struct S {
    int f();
    static bool g();
    int h(int);
    int (*fp)();
};
int f(int);
int h(S, char *);
int h(S, int *);

int x1 = f(1);      // ok, calls ::f(int)
int x2 = f(S());    // ok, equivalent to S().f()
int x3 = ::f(S());  // error: cannot convert S to int
int x3a = (f)(S()); // error: cannot convert S to int
bool x4 = g(S());   // ok, equivalent to S().g()
int x5 = h(S(), 0); // error: ambiguous; does not consider S::h(int)
int x6 = fp(S());   // ok, equivalent to S().fp()
```

-- end example]

If the selected function is non-virtual, or if the *id-expression* in the class member access expression is a *qualified-id*, that function is called. Otherwise, its final overrider (10.3) in the dynamic type of the object expression is called; such a call is referred to as a virtual function call. [Note: the dynamic type is the type of the object referred to by the current value of the object expression. 12.7 describes the behavior of virtual function calls when the object expression refers to an object under construction or destruction. -- end note]